Department of Computer Engineering

CENG350 Software Engineering

Software Requirements Specification for Koster Seafloor Observatory (KSO)

Group # 5

By

Deniz Karakoyun

Alper Mehmet Celebi

Thursday 10th April, 2025

Contents

List of Figures

List of Tables

Revision History

(Clause 9.2.1)

You can use a table to show your reports' versions and their date.

To create tables, you can use https://www.latex-tables.com/; the Table 1 is generated by it.

A	B	C	D

Table 1: Table Example

1. Introduction

1.1 Purpose of the System

The Koster Seafloor Observatory (KSO) software is designed to facilitate the monitoring, analysis, and visualization of subsea environments within Kosterhavets National Park. By integrating open-source tools, citizen science, and machine learning, KSO enhances marine biodiversity research by streamlining the processing and analysis of underwater video data. The software enables researchers to efficiently assess ecosystem changes, detect species, and contribute to long-term marine conservation efforts. It addresses key challenges in data management, annotation, and automated image analysis, improving the efficiency, accuracy, and scalability of marine ecosystem studies through advanced object detection and classification techniques.

1.2 Scope

The Koster Seafloor Observatory is an open-source modular platform designed to process and analyze subsea movie data. Marine researchers face significant challenges in handling vast amounts of subsea imagery, including the efficient management of large video datasets, the time-consuming and error-prone task of manual annotation, and the need for advanced computational tools for automating the detection and classification of marine species.

The KSO system has been designed to move and process underwater footage and its associated data (e.g., location, date, sampling device), make this data available

to citizen scientists via Zooniverse for annotation, and train and evaluate machine learning models (customizing Yolov5 or Yolov8 models). The system is structured around Jupyter Notebooks, which allow users to perform specific tasks such as uploading footage to the citizen science platform or analyzing classified data. These notebooks can be run via Google Colab, locally, or in a high-performance computing (HPC) environment.

This software contributes significantly to marine biodiversity research by providing scalable, automated data processing for underwater imagery. By integrating with existing research infrastructures and machine learning workflows, it enhances data accessibility and ensures the accuracy of marine ecological studies. The software is interoperable with major open-source tools and platforms, making it highly adaptable for research and conservation initiatives.

What the KSO System Is:

- A system for managing, archiving, and analyzing underwater video footage.
- A platform for citizen scientists to contribute annotations to marine ecological datasets.
- A tool for training and evaluating machine learning models for species detection.
- An integration of Jupyter Notebooks for accessible, structured data processing.
- A scalable, open-source solution that supports marine biodiversity research.

What the KSO System Is Not:

- A real-time monitoring or surveillance system for underwater environments.
- A substitute for human-expert ecological analysis but rather a supporting tool.
- A proprietary, closed-source system; it is open and collaborative.
- A hardware-dependent software; it supports cloud, local, and HPC environments.

Primarily applied in marine ecological research—especially in monitoring cold-water coral ecosystems in Sweden's Kosterhavets National Park—the Observatory offers several key benefits. Its modular design ensures scalability and adaptability to various marine datasets, while automation provided by machine learning algorithms reduces the reliance on manual annotation. Additionally, integrating citizen scientists fosters collaboration and increases public engagement in biodiversity monitoring, and the open-source tools along with the API integration promote broader adoption and customization within the research community.

1.3 System Overview

1.3.1 System Perspective

The Koster Seafloor Observatory (KSO) is an integral part of a broader marine research ecosystem, working in close integration with existing research institutions and open-source platforms. The system is designed to process, analyze, and store vast amounts of underwater footage while leveraging the collective efforts of citizen scientists and artificial intelligence for marine biodiversity monitoring. The system's workflow follows a structured pipeline consisting of data ingestion, annotation, machine learning training, and ecological insights generation.

The KSO framework is built upon Jupyter Notebooks, which serve as a modular and user-friendly environment for performing various tasks, including uploading footage, annotating images, and evaluating machine learning models. The system can be executed in different computational environments, including Google Colab, local machines, and high-performance computing (HPC) clusters, ensuring accessibility and scalability.

The **context diagram** illustrates the high-level interactions between the **KSO System** (the core software platform) and its external entities. It shows how different user groups and subsystems interact with KSO, as well as the flow of data between them. The context diagram consists of different parts. Related explanations are:

1. **Researchers:** Researchers are primary stakeholders who:

- **Upload and analyze underwater video data** to the KSO system.
- **Receive processed and annotated footage** enriched with species classifications.
- **Use the system's outputs** to draw ecological insights and support marine biodiversity research.

2. **Citizen Scientists:** Citizen scientists contribute to the system by:

- **Receiving video footage for annotation** via the KSO interface.
- **Identifying and labeling species** visible in the footage.
- Their inputs are used to **train machine learning models** and improve annotation reliability.

3. **Machine Learning System:** The machine learning component:

- **Receives annotated footage and labels** from KSO to **train species classification models**.
- **Generates automated species classifications** for new video data.
- It interfaces with the KSO system via an API, implemented using **FastAPI**, for real-time inference and result delivery.

4. **External Databases:** External databases interact with KSO to:

- **Supply archived underwater video footage** for processing and analysis.
- **Store annotated and classified footage** produced by the system for long-term use and integration with biodiversity data portals.

5. **KSO System:** KSO is the central platform that:

- Manages **data ingestion, annotation workflows**, and **machine learning integration**.
- Coordinates the **interaction between researchers, citizen scientists, ML models, and data repositories**.
- Facilitates **data aggregation, processing, and interface delivery** through a web-based UI and backend API services.

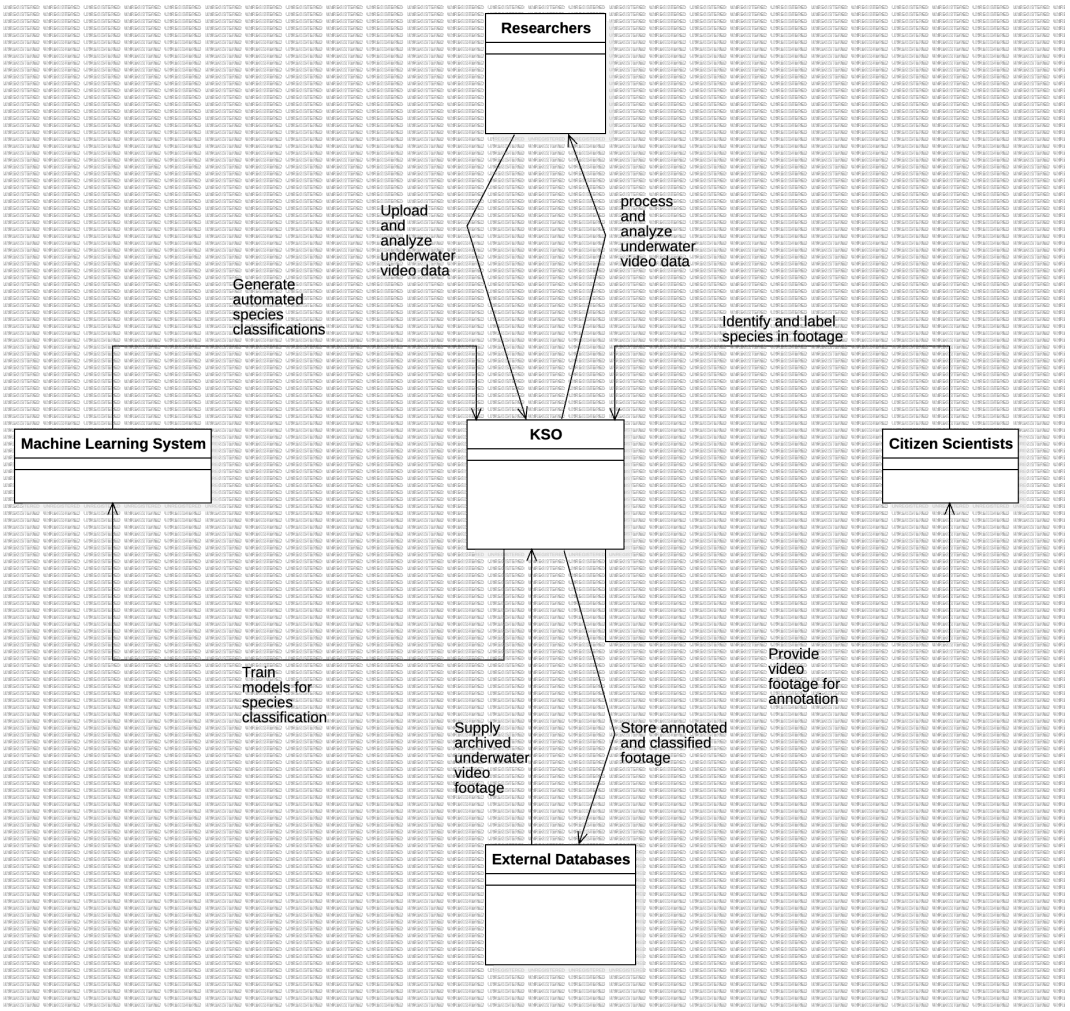

Figure 1.1: Context Diagram

1.3.1.1 System Interfaces

- **Data Management Interface:** Acts as the backbone of the system, responsible for storing, retrieving, and preprocessing underwater footage and metadata (e.g., location, sampling device, and timestamp).
- **Annotation Interface:** A web-based platform (e.g., Zooniverse) that allows citizen scientists to contribute annotations by identifying species and habitat features in video frames.
- **Machine Learning Interface:** Facilitates the training, testing, and deployment of deep-learning models (e.g., Yolov5 and Yolov8) for automated species detection and classification.
- **Visualization and Analysis Interface:** Provides tools for researchers to analyze annotated data, visualize detected species distributions, and extract ecological insights.

1.3.1.2 User Interfaces

The KSO software is designed with multiple user categories in mind, each with specific interaction needs:

- **Citizen Scientists:** Engage with the system via an interactive web-based annotation tool to classify marine species in underwater footage.

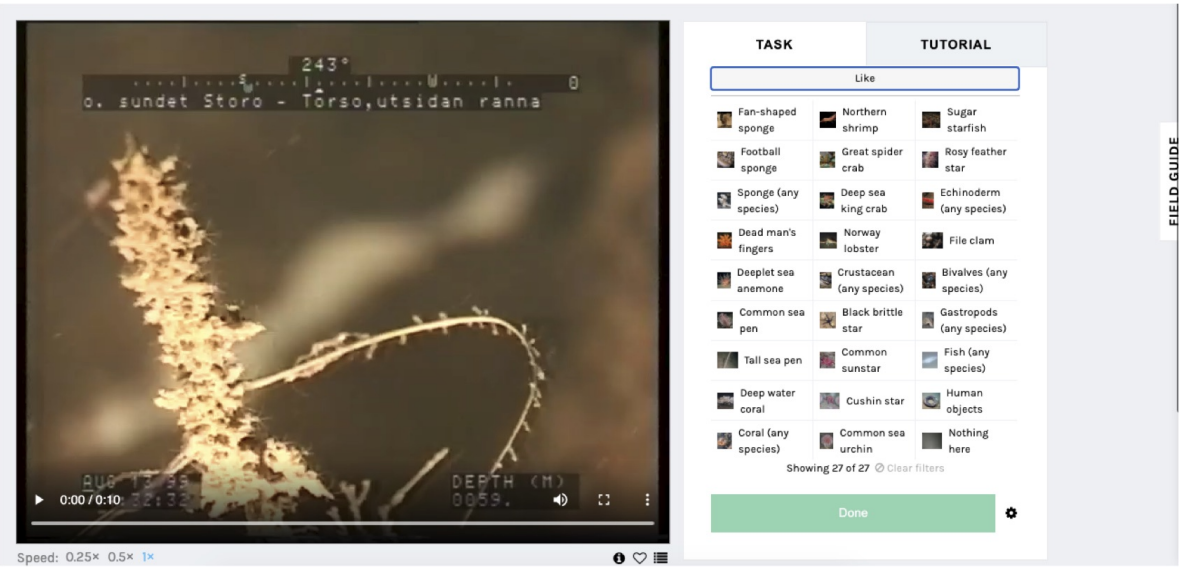

Figure 1.2: Web-based annotation interface from Zooniverse used by Citizen Scientists.

Figure 1.3: Web-based annotation interface example annotated rectangles provided by a citizen scientist.

- **Researchers and Marine Biologists:** Use Jupyter Notebooks and visualization tools to analyze annotated data, extract biodiversity patterns, and train machine learning models.
- **Developers and Data Scientists:** Work with the machine learning pipeline to enhance species detection algorithms and integrate new deep-learning models.

1.3.1.3 Hardware Interfaces

- Compatibility with underwater camera systems and ROVs
- Data processing on high-performance computing (HPC) clusters or cloud-based services

1.3.1.4 Software Interfaces

KSO integrates various external software and libraries for data processing and machine learning operations:

- **Storage and Data Management:** Uses cloud-based storage solutions to store and retrieve video datasets.
- **Machine Learning Frameworks:** Leverages OpenCV for deep learning-based image classification and object detection.
- **API and Data Integration:** Provides APIs to fetch and process annotation data from external platforms like Zooniverse and Ocean Data Factory Sweden.
- **Model Deployment and Accessibility:** The trained machine learning model is deployed through an application programming interface (API) using FastAPI, ensuring high-speed, scalable, and reliable inference for multiple users simultaneously.
- **User-Friendly Frontend:** The API is integrated with a Streamlit-based frontend, allowing researchers, ecologists, ROV and AUV pilots, and students to access the service via a web application.
- **Interactive Model Interface:** The frontend allows users to browse classified footage, upload new images or videos, adjust hyperparameter thresholds (e.g., IOU threshold, confidence threshold), and see real-time changes in model outputs.

- **Containerized Deployment:** The Koster app module uses Docker Compose to manage backend and frontend services efficiently, ensuring compatibility and ease of deployment across different environments.

1.3.1.5 Communications Interfaces

The system enables seamless communication across different components using well-defined protocols:

- **HTTP/HTTPS Protocols:** Secure communication for data exchange between the annotation platform, cloud storage, and processing servers.
- **RESTful APIs:** Enables integration with external research databases and real-time data retrieval.
- **Messaging and Logging:** Uses messaging queues and logging frameworks to track processing pipelines and ensure system transparency.

1.3.1.6 Memory Constraints

Given the large size of video datasets, the system optimizes storage and memory allocation:

- High-resolution video data is stored efficiently using compression algorithms.
- Temporary caching techniques are applied to accelerate real-time annotation and model inference.
- Distributed storage solutions enable scalability for long-term ecological studies.

1.3.1.7 Operations

The KSO system supports multiple operational workflows:

- Uploading and managing underwater footage for research analysis.
- Enabling citizen scientists to annotate video frames through an online platform.

- Training machine learning models to improve automated species classification.
- Visualizing and interpreting marine biodiversity data through interactive dashboards.

By leveraging open-source technologies, citizen science, and machine learning, the Koster Seafloor Observatory software provides an innovative and scalable solution for monitoring marine biodiversity.

1.3.2 System Functions

The following table outlines the key functionalities of the Koster Seafloor Observatory and their descriptions.

Function	**Description**
Extract Ecological Data	This function allows for the extraction of spatiotemporal distribution and relative abundance of habitat-building species from deep-water recordings in the Kosterhavets National Park.
Citizen Science Module	This function enables citizen scientists and researchers to collaboratively annotate raw underwater footage for species identification and location.
Species Identification Workflow	In this function, citizen scientists classify biological objects in underwater footage by selecting species or animal groups, indicating their number, and noting the time they appear.
Object Location Workflow	This function allows citizen scientists to locate biological objects within still underwater images, aiding in the classification process.

Function	Description
Identify Benthic Species Distribution	This function helps in identifying the distribution and abundance of benthic species through underwater footage analysis.
Access Supporting Materials	This function provides access to supporting materials like tutorials, background information, and field guides for citizen scientists involved in species identification and location.
Generate EBV Data Products	This function generates Essential Biodiversity Variables (EBVs) based on the analysis of underwater footage, which can be used for broad spatial and temporal scale analyses.
Facilitate National Monitoring	This function supports national monitoring programs by providing ecological data products from image-based sensors, especially in marine environments.
Collaborate with Research Institutes	This function enables collaboration between multiple research institutes for analyzing underwater footage data and deriving useful ecological insights.
Integrate Data for Large-Scale Analysis	This function enables large-scale analysis by aggregating and processing data from underwater recordings collected by various research institutions, ensuring broader ecological insights.

1.3.3 Stakeholder Characteristics

This section provides an overview of the general characteristics of the intended user groups for the KSO software. These characteristics—ranging from educational background to technical expertise—are described here to explain why specific requirements

are subsequently specified. No specific requirements are stated in this section; rather, the focus is on how stakeholders' varying needs influence the design and usability considerations for the system.

- **Citizen Scientists**
  - *Education and Experience:* May range from high school students to retired professionals, many with minimal formal training in marine biology or data analysis.
  - *Motivation:* Typically driven by an interest in marine conservation; benefit from clear guidance, intuitive user interfaces, and engaging tasks to effectively contribute to annotations.
  - *Accessibility Considerations:* Could include individuals with limited digital literacy or physical impairments, so features like screen readers, clear text labeling, and simple workflows are essential.
- **Researchers and Marine Biologists**
  - *Education and Experience:* Usually possess advanced degrees in marine science or related fields; many have robust backgrounds in ecological research, data collection, and scientific analysis.
  - *Technical Expertise:* Comfortable using analytical tools and may have moderate programming skills; prefer interfaces that reduce tedious tasks like manual data upload and annotation checks.
  - *Usability Influences:* Benefit from comprehensive visualization, reporting, and analytics capabilities to derive insights quickly and accurately.
- **Developers and Data Scientists**
  - *Technical Expertise:* Skilled in programming, machine learning, and data engineering; regularly work with environments like Jupyter Notebooks, HPC clusters, and cloud-based solutions.

  - *Focus Areas:* Require high performance, scalability, and extensibility; emphasize transparency in workflows (e.g., open-source frameworks, version control) for easy collaboration.
  - *Collaboration Needs:* Rely on robust APIs, logging, and modular architectures to integrate new deep-learning models or enhancements.

- **Project Coordinators**
  - These stakeholders oversee the overall operation of the Koster Seafloor Observatory, ensuring that the project objectives are met and that all stakeholders collaborate effectively. They manage communication between researchers, citizen scientists, and technical staff, and they are responsible for the strategic direction of the project, including securing funding and partnerships.

By outlining these stakeholder characteristics, the KSO software can be tailored to address distinct usability and accessibility needs, ensuring that each group finds the system both approachable and effective. Recognizing these user profiles also clarifies why particular technical and interface requirements appear in later sections of this document.

1.3.4 Limitations

This section describes constraints and limitations that may restrict the supplier's options during the development and deployment of the Koster Seafloor Observatory (KSO) system. These limitations are derived from a variety of sources, including regulatory requirements, hardware/software constraints, and interoperability needs.

- **Regulatory Requirements and Policies:** The KSO system must comply with applicable regional and international regulations regarding data privacy, environmental standards, and research ethics (e.g., GDPR for data protection when collecting user information). Any data contributed by citizen scientists or researchers must be handled according to relevant consent and confidentiality guidelines.

- **Hardware Limitations (Signal Timing Requirements):** While primarily software-based, the system may depend on video recording devices and underwater camera systems whose performance (e.g., frame rate, resolution) can affect data processing. These devices may have inherent signal timing constraints (e.g., capturing high-resolution video feeds with limited bandwidth).

- **Interfaces to Other Applications:** The KSO system integrates external services such as cloud storage providers (e.g., AWS S3, Google Drive) and annotation platforms (e.g., Zooniverse). Any changes in these third-party APIs, terms of service, or data formats can limit the system's interoperability and may require amendments to the codebase.

- **Parallel Operation:** Parallel processing on high-performance computing (HPC) clusters or cloud-based infrastructure depends on cluster resource availability, queue scheduling, and job prioritization. This can limit throughput and real-time data processing capacity.

- **Audit Functions:** Auditing user annotations and machine learning model outputs necessitates robust logging mechanisms and version control. Absence or misconfiguration of these features can hinder traceability and compliance with research data management standards.

- **Control Functions:** Control over how and when data is ingested or processed may be subject to research protocols, HPC job scheduling policies, or data access permissions. This can reduce flexibility in adapting workflows or rapidly iterating on model configurations.

- **Higher-Order Language Requirements:** The system's Jupyter Notebook-based workflows are primarily implemented in Python, leveraging libraries such as PyTorch, TensorFlow, and OpenCV. Availability of updated packages and compatibility considerations with the host environment can constrain software upgrades and feature adoption.

- **Signal Handshake Protocols:** Direct signal-level handshake protocols between the KSO software and hardware devices (e.g., underwater cameras) are typically managed by lower-level drivers or external data-acquisition systems. Any constraints related to streaming data reliability may limit the speed and volume of footage ingestion.

- **Quality Requirements (Reliability):** Managing large volumes of high-resolution video data necessitates robust storage solutions, backup strategies, and error-handling routines. Failures in data monitoring or storage can limit the reliability of both the system and downstream analytical results.

- **Criticality of the Application:** While the system is not designated as a safety-critical or life-critical application, research accuracy and data integrity remain crucial. Incorrect model outputs or misinterpretation of ecosystem changes could negatively impact conservation initiatives and research conclusions.

- **Safety and Security Considerations:** The KSO software handles potentially sensitive location data (e.g., coordinates for ecological studies). Ensuring secure communication channels (HTTPS/SSL) and strict access controls is essential to prevent unauthorized use of sensitive marine habitat information.

- **Physical/Mental Considerations:** Citizen scientists and researchers may have differing physical abilities or levels of experience navigating complex data tools. Interfaces must consider usability aspects such as accessible design features to facilitate broad participation and minimize user fatigue.

- **Limitations Sourced from Other Systems (Real-Time Requirements):** The KSO system is designed for batch processing and offline analyses, rather than real-time event monitoring. Users requiring near-instantaneous data processing (e.g., real-time habitat surveillance) may need to adopt specialized solutions outside the scope of the current software. In cases where data sharing is required from real-time ROV operations or other research instruments, the system may

be constrained by the throughput and data formats provided by those external systems.

These limitations collectively shape the technical and operational boundaries within which the KSO software can function. Adhering to these constraints during design and development helps ensure that the system remains reliable, compliant, and aligned with stakeholder needs.

1.4 Definitions

Term	Definition
ROV (Remotely Operated Vehicle)	An underwater robot used to capture video footage and data from marine environments.
AUV (Autonomous Underwater Vehicle)	An autonomous robot used for underwater survey missions, capturing data without direct human control.
HPC (High-Performance Computing)	A form of computing that uses supercomputers and computer clusters to solve advanced computation problems.
API (Application Programming Interface)	A set of protocols and tools for building software and applications, allowing different systems to communicate.
YOLO (You Only Look Once)	A real-time object detection system that applies a single neural network to the full image.
Zooniverse	A citizen science web portal that allows volunteers to participate in scientific research by annotating data.
EBV (Essential Biodiversity Variables)	A set of measurements that capture key dimensions of biodiversity change.
GDPR (General Data Protection Regulation)	A regulation in EU law on data protection and privacy in the European Union and the European Economic Area.
SSL (Secure Sockets Layer)	A standard security technology for establishing an encrypted link between a server and a client.
IoU (Intersection over Union)	A metric used to evaluate the accuracy of an object detector on a particular dataset.
mAP (mean Average Precision)	A metric used to evaluate the accuracy of object detection models.
OpenCV (Open Source Computer Vision Library)	An open-source computer vision and machine learning software library.
PyTorch	An open-source machine learning library based on the Torch library, used for applications such as computer vision and natural language processing.
TensorFlow	An open-source software library for dataflow and differentiable programming across a range of tasks, used for machine learning applications.

Table 1.2: Definitions of Key Terms

2. References

[1] Wildlife.ai, "Koster observatory project." Online, 2023. Available: https://wildlife.ai/projects/koster-observatory/.

[2] O. D. F. Sweden, "Kso github repository." GitHub, 2023. Available: https://github.com/ocean-data-factory-sweden/kso.

[3] O. D. F. Sweden, "Koster ml github repository." GitHub, 2023. Available: https://github.com/ocean-data-factory-sweden/koster_ml.

[4] V. Anton, J. Germishuys, P. Bergström, M. Lindegarth, and M. Obst, "An open-source, citizen science and machine learning approach to analyse subsea movies," *Biodiversity Data Journal*, vol. 9, p. e60548, 2021. Available: https://doi.org/10.3897/BDJ.9.e60548.

[5] P. Publishers, "Biodiversity data journal: Koster observatory article." Online, 2021. Available: https://bdj.pensoft.net/article/60548/element/5/6667321//.

[6] S. Project, "Documentation for underwater monitoring systems." Online, 2023. Available: https://subsim.se/documentation.

[7] IEEE, "Ieee standard 29148-2018: Systems and software engineering - life cycle processes - requirements engineering," 2018.

[8] Zooniverse, "Citizen science platform for marine research." Online, 2023. Available: https://www.zooniverse.org.

3. Specific Requirements

3.1 External Interfaces

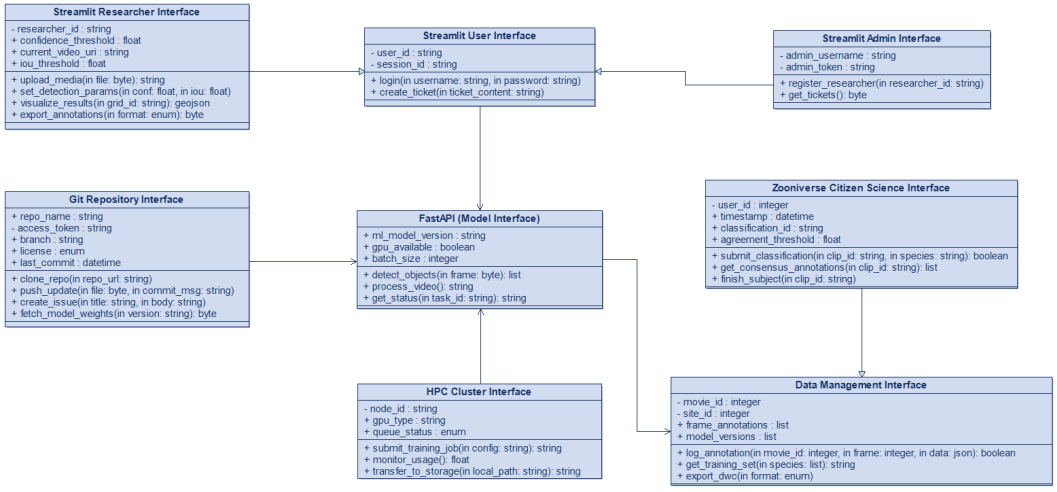

Figure 3.1: External Interfaces Diagram

Explanations of Interfaces

1. Streamlit Researcher Interface

Purpose: Allows researchers to upload media, adjust detection parameters, and visualize results.

Key Fields:

- `confidence_threshold`: Minimum confidence score for object detection (e.g., 0.7 for cold-water corals).
- `iou_threshold`: Overlap threshold for bounding box merging.

Functions:

- `upload_media`: Upload videos/images for analysis.
- `visualize_results`: Render detections as GeoJSON for mapping (e.g., coral distributions in Fig. 6 of the paper).
- `export_annotations`: Export data in formats like CSV or Darwin Core.

2. Zooniverse Citizen Science Interface

Purpose: Integrates with Zooniverse for crowd-sourced annotations.

Key Fields:

- `agreement_threshold`: Consensus threshold (e.g., 80 percent of citizen scientists must agree).

Functions:

- `submit_classification`: Log species identifications from citizen scientists.
- `get_consensus_annotations`: Aggregate annotations (e.g., retain overlapping bounding boxes).

3. Git Repository Interface

Purpose: Version control for code, models, and data.

Key Fields:

- `repo_name`: GitHub repository (e.g., koster_ml).
- `access_token`: Authentication for pushing/pulling updates.

Functions:

- `fetch_model_weights`: Download trained models (e.g., YOLOv3 weights).
- `push_update`: Commit new code/data to the repository.

4. FastAPI (Model Interface)

Purpose: Hosts the machine learning model as an API.

Key Fields:

- `ml_model_version`: Model version (e.g., "YOLOv3_Koster_1.0").
- `gpu_available`: A boolean attribute indicating if there is sufficent GPU available.

Functions:

- `detect_objects`: Run object detection on a single frame.
- `process_video`: Batch-process videos (e.g., extract coral observations).

5. HPC Cluster Interface

Purpose: Handles resource-intensive tasks like model training.

Key Fields:

- `gpu_type`: GPU specification.
- `queue_status`: Job status (e.g., "running", "completed").

Functions:

- `submit_training_job`: Train models using annotated data.
- `monitor_usage`: Track CPU/GPU utilization.

6. Data Management Interface

Purpose: Manages raw data, annotations, and metadata.

Key Fields:

- `frame_annotations`: Stores bounding boxes from citizen scientists/ML.
- `model_versions`: Tracks model iterations.

7. Streamlit Admin Interface

Purpose: Manages Streamlit interface, fixes researchers' problem by checking tickets created with the platform.

Key Fields:

- `admin_username`: Unique username for admin.
- `admin_token`: Unique token for admin.

Functions:

- `register_researcher`: Adding new researcher to the system.
- `get_tickets`: Retrieve tickets to solve users' problems.

8. Streamlit User Interface

Purpose: Main interface for researchers and admins in the system.

Key Fields:

- `login`: Logging in the system.
- `create_ticket`: Creates ticket for specified user problems about the system.

Functions:

- `register_researcher`: Adding new researcher to the system.
- `get_tickets`: Retrieve tickets to solve users' problems.

3.2 Functions

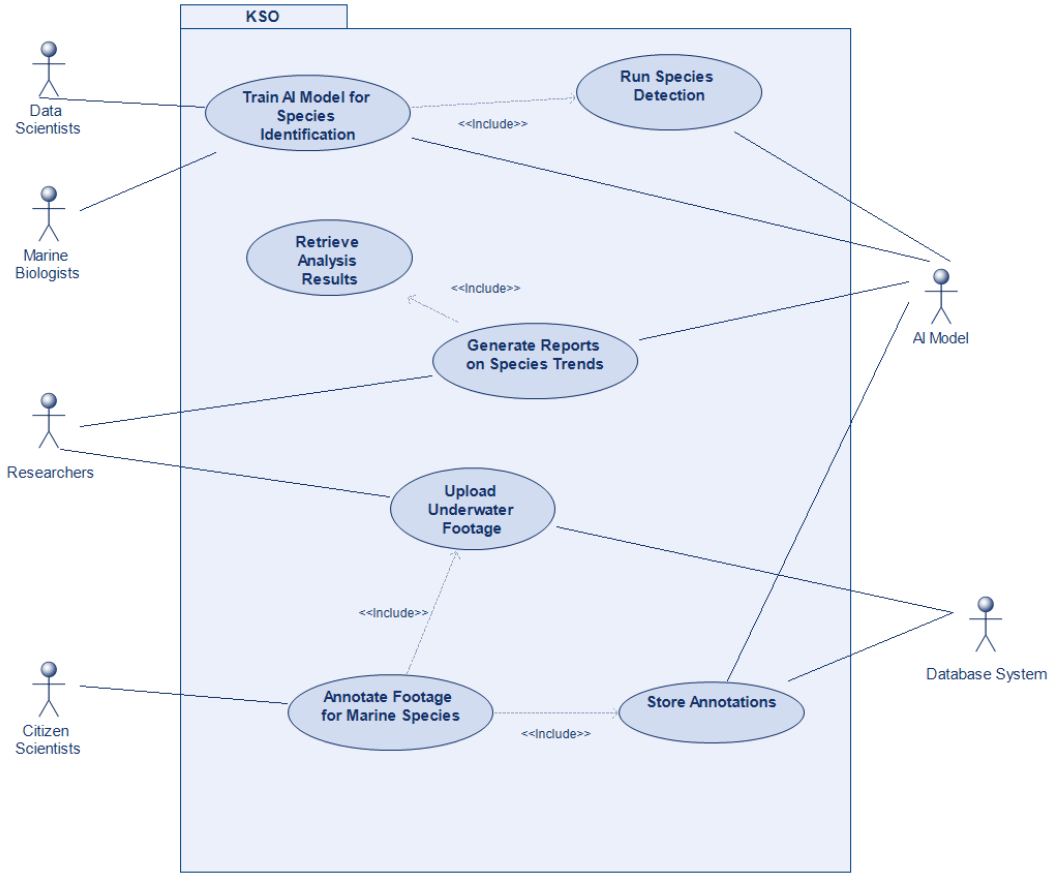

Figure 3.2: Use Case Diagram

Table 3.1: Use Case: Annotate Footage for Marine Species

Use Case Name	**Annotate Footage for Marine Species**
Actors	Researchers, Citizen Scientists
	Continued on next page

Table 3.1 – continued from previous page

Use Case Name	**Annotate Footage for Marine Species**
Description	Researchers upload marine footage to KSO, which citizen scientists annotate. The AI model later uses these annotations for training.
Pre-Conditions	- Footage must be available. - Citizen scientists must have platform access. - AI model requires a sufficient number of labeled images.
Data	- Uploaded marine footage - Annotated species data - AI model training dataset - User annotation logs
Response	- System acknowledges footage upload. - System provides annotation tools for users. - System validates and stores annotations. - AI model retrains based on new annotations.
Stimulus	- Researchers initiate footage upload. - Citizen scientists start annotating footage. - AI model processes new labeled data.

Continued on next page

Table 3.1 – continued from previous page

Use Case Name	**Annotate Footage for Marine Species**
Normal Flow	1. Researchers upload marine footage to KSO. 2. System stores footage in a database. 3. Citizen scientists access footage through the annotation interface. 4. Users label species in the footage. 5. Annotations are submitted and stored in the database. 6. System verifies annotation accuracy (automated + expert review). 7. Valid annotations are added to the training dataset. 8. AI model uses annotations to improve species recognition. 9. System updates annotation status.
Alternative Flow	- If no citizen scientists are available, researchers manually annotate footage. - AI model suggests labels based on existing training data, which users approve or correct.
Exception Flow	- If footage upload fails, system prompts researcher to retry. - If annotation submission fails, user receives an error message and can resubmit. - If AI model accuracy is below threshold, additional manual review is triggered.
Post-Conditions	- Annotated footage is successfully stored. - AI model is updated with new labeled data. - System generates reports on annotation progress.

Continued on next page

Table 3.1 – continued from previous page

Use Case Name	**Annotate Footage for Marine Species**
Comments	- Annotations are periodically reviewed by marine biology experts. - Users with a high accuracy rating gain advanced annotation privileges. - System maintains a history of all annotations for research purposes.

Table 3.2: Use Case: Generate Reports on Species Trends

Use Case Name	**Generate Reports on Species Trends**
Actors	Researchers, AI Model
Description	The system compiles historical data to generate insights into species population changes, migration patterns, and environmental impacts.
Pre-Conditions	- Historical and recent observation data must be available. - AI model trained on species recognition and trend analysis. - Organizations require access to reports.
Data	- Annotated species data - Migration records - Environmental condition reports - AI-generated trend analyses
Response	- System generates species trend reports. - Reports highlight critical population changes. - Users can filter data by species, location, and time.

Continued on next page

Table 3.2 – continued from previous page

Use Case Name	**Generate Reports on Species Trends**
Stimulus	- Researchers request species reports. - AI model identifies emerging trends.
Normal Flow	1. System compiles species data from multiple sources. 2. AI analyzes population changes and correlations. 3. Reports are generated and visualized. 4. Researchers interpret data and publish findings.
Alternative Flow	- If dataset is incomplete, system estimates missing values. - Manual validation ensures report accuracy.
Exception Flow	- If species data is unreliable, the system flags inconsistencies. - Users can provide corrections or additional inputs.
Post-Conditions	- Reports are interpreted with researchers. - Data informs policy strategies. - AI model updates based on new inputs.
Comments	- Reports can be exported in multiple formats. - Real-time dashboards allow users to track changes dynamically.

Table 3.3: Use Case: Train AI Model for Species Identification

Use Case Name	**Train AI Model for Species Identification**
Actors	AI Model, Data Scientists, Marine Biologists, Researchers
	Continued on next page

Table 3.3 – continued from previous page

Use Case Name	**Train AI Model for Species Identification**
Description	The AI model is trained using annotated footage of marine species, improving its accuracy in recognizing different species in newly collected data.
Pre-Conditions	- A sufficient dataset of labeled species images must be available. - Data scientists must have access to the training pipeline. - System must be capable of iterative training and validation.
Data	- Labeled marine species dataset - AI model training logs - Expert-verified annotations - Model performance metrics
Response	- System processes annotated footage for model training. - AI model is updated with new training data. - System evaluates the model's performance. - Researchers and biologists validate model outputs. - Updated AI model is deployed for species identification.
Stimulus	- New annotated species data is uploaded. - AI model training is triggered. - System evaluates model accuracy and performance.

Continued on next page

Table 3.3 – continued from previous page

Use Case Name	**Train AI Model for Species Identification**
Normal Flow	1. Annotated species data is collected. 2. AI model training is initiated using the new dataset. 3. System processes and optimizes training iterations. 4. Model performance is evaluated on test data. 5. Experts review and validate the AI model's predictions. 6. If accuracy is sufficient, the model is deployed for real-world species identification.
Alternative Flow	- If labeled data is insufficient, data augmentation techniques are applied. - If the model does not meet accuracy thresholds, additional expert validation is required.
Exception Flow	- If training data is corrupt or insufficient, the system halts training and requests additional labeled data. - If model accuracy is too low, retraining with refined parameters is initiated.
Post-Conditions	- AI model is successfully updated with new labeled data. - Model achieves higher accuracy in species recognition. - Researchers gain insights into model performance and areas for improvement.
Comments	- AI model training is continuously refined through periodic updates. - Researchers contribute expert-labeled data to improve performance. - System keeps track of model versioning for reproducibility.

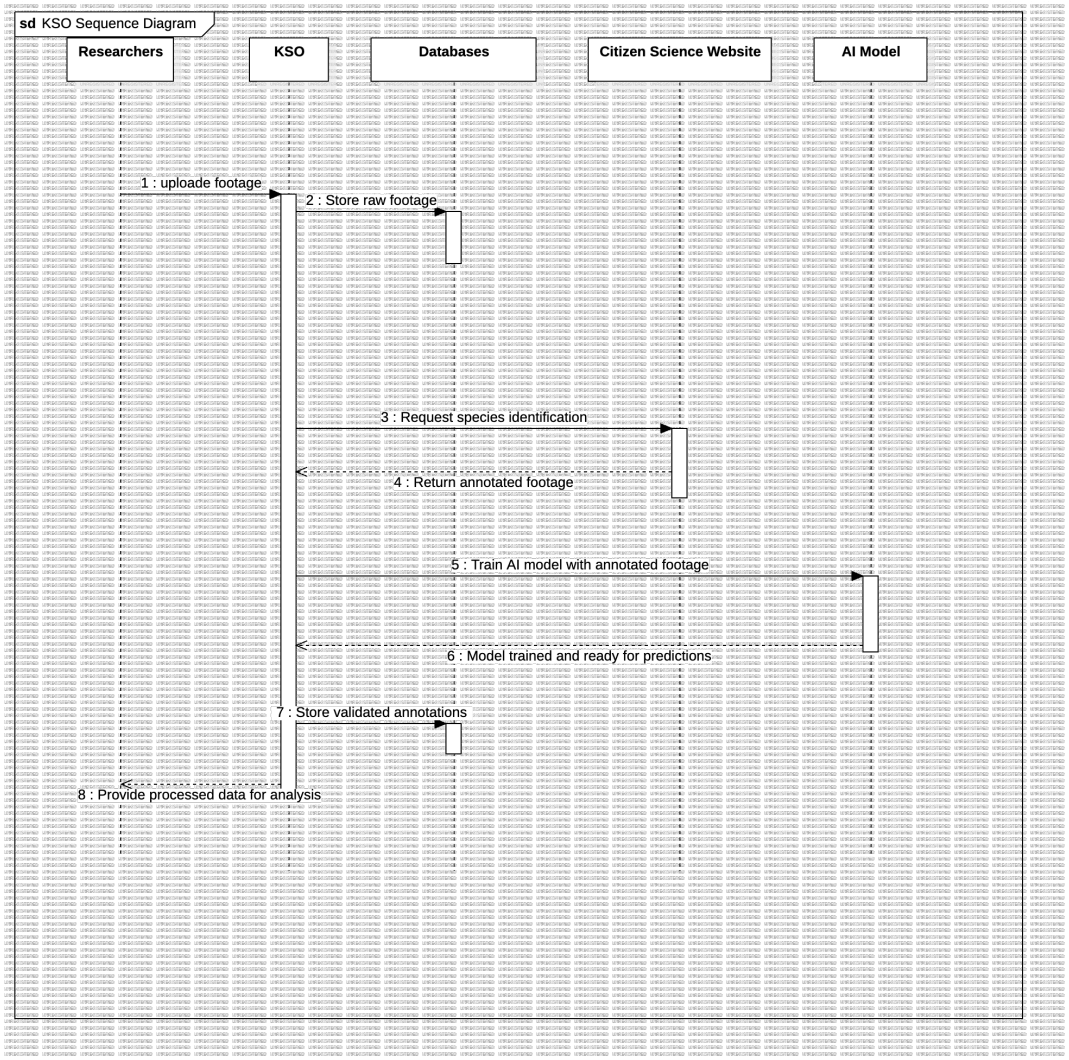

Figure 3.3: Sequence Diagram for Annotate Footage for Marine Species

Table 3.4: Use Case: Upload Underwater Footage

Use Case Name	**Upload Underwater Footage**
Actors	Researchers, Database System
Description	Researchers upload raw underwater footage (ROV/AUV recordings) to the KSO system for storage and subsequent analysis.
	Continued on next page

Table 3.4 – continued from previous page

Use Case Name	**Upload Underwater Footage**
Pre-Conditions	- Researcher has valid credentials - Footage meets format requirements (e.g., MP4/MOV) - Storage space is available
Data	- Raw video files - Metadata (timestamp, GPS coordinates, depth) - Researcher credentials
Response	- System validates file format and metadata - Stores footage in hot/cold storage - Updates database records - Returns upload confirmation
Stimulus	Researcher initiates upload via Streamlit interface
Normal Flow	1. Researcher selects video files for upload 2. System verifies file format (e.g., checks extension) 3. Researcher adds metadata (site ID, collection date) 4. System generates unique movie_id 5. Footage is encrypted and transferred to short-term storage 6. Database records metadata in `movies` table 7. System logs upload event 8. Researcher receives success notification
Alternative Flow	- If file size is greater than 1GB, system queues for batch upload - If metadata incomplete, prompts for mandatory fields
Exception Flow	- If storage full: notifies admin and pauses upload - If invalid format: rejects file with error details - If connection fails: resumes transfer from last chunk
Continued on next page	

Table 3.4 – continued from previous page

Use Case Name	**Upload Underwater Footage**
Post-Conditions	- Footage is searchable in database - Storage usage metrics updated - Processing pipeline triggered (if auto-queue enabled)
Comments	- Files longer than 2 hours are automatically routed to cold storage - MD5 checksum generated for integrity verification - Sensitive locations may require access approval

Table 3.5: Use Case: Store Annotations

Use Case Name	**Store Annotations**
Actors	Database System, Citizen Scientists, AI Model
Description	Records species classification data from both human annotators and ML predictions in the KSO database.
Pre-Conditions	- Annotations pass validation checks - Database is accessible - Associated movie/species IDs exist
Data	- Raw annotations (bounding boxes, species labels) - Confidence scores (ML) - Annotator metadata (user_id, timestamp)
Response	- Writes to database tables - Updates table retirement status - Returns storage confirmation
Stimulus	Annotation submission via Zooniverse API or ML batch results

Continued on next page

Table 3.5 – continued from previous page

Use Case Name	**Store Annotations**
Normal Flow	1. System receives annotation payload 2. Validates required fields 3. For citizen science: checks agreement threshold 4. For ML: logs confidence score and model version 5. Generates database records with timestamps 6. Updates related tables 7. Triggers consensus calculation if needed
Alternative Flow	- Low-agreement citizen annotations route to expert review - ML annotations below confidence threshold are flagged for retraining
Exception Flow	- If DB write fails: retries 3x then queues for async processing - If foreign key violation: logs orphaned annotations for recovery
Post-Conditions	- Annotations are queryable via API - Training sets automatically update - Retired subjects are archived
Comments	- Uses atomic transactions for data integrity - Maintains provenance (human vs. ML source) - Supports Darwin Core transformation

Table 3.6: Use Case: Run Species Detection

Use Case Name	**Run Species Detection**
Actors	Machine Learning System
	Continued on next page

Table 3.6 – continued from previous page

Use Case Name	**Run Species Detection**
Description	Executes trained object detection models to identify and classify marine species in newly uploaded footage.
Pre-Conditions	- Model weights are loaded - Input footage is preprocessed - Detection thresholds are configured
Data	- Video frames - Model configuration - Confidence/IoU parameters
Response	- Bounding box coordinates - Species classification labels - Confidence scores per detection
Stimulus	New footage availability or researcher request
Normal Flow	1. System ingests preprocessed frames 2. Performs forward pass through model 3. Applies non-max suppression 4. Filters detections below threshold 5. Formats results for storage 6. Writes to model_annotations table
Alternative Flow	- Low-confidence detections route for manual review - Unrecognized species trigger model retraining
Exception Flow	- Model failure initiates rollback - Invalid input format halts processing
Post-Conditions	- Detections are stored - Model performance metrics updated - Results available for retrieval

Continued on next page

Table 3.6 – continued from previous page

Use Case Name	**Run Species Detection**
Comments	- Supports batch and real-time modes - Thresholds configurable per species

Table 3.7: Use Case: Retrieve Analysis Results

Use Case Name	**Retrieve Analysis Results**
Actors	Researchers
Description	Queries stored detection results through the API for visualization or export.
Pre-Conditions	- Analysis job is complete - Researcher has permissions - Results exist for query parameters
Data	- Query filters (time range, species, location) - Authentication tokens - Output format preference
Response	- Annotated frames or videos - Tabular detection data - Aggregated statistics
Stimulus	Researcher API request
Normal Flow	1. Researcher submits query via API 2. System validates permissions 3. Retrieves data from model_annotations 4. Applies requested transformations 5. Returns formatted results
Alternative Flow	- Large result sets trigger pagination - Complex queries execute as async jobs

Continued on next page

Table 3.7 – continued from previous page

Use Case Name	**Retrieve Analysis Results**
Exception Flow	- Invalid query returns error codes - Missing data prompts reanalysis request
Post-Conditions	- Access logs updated - Results cached for subsequent queries
Comments	- Supports GeoJSON and DwC outputs - Rate-limited for system stability

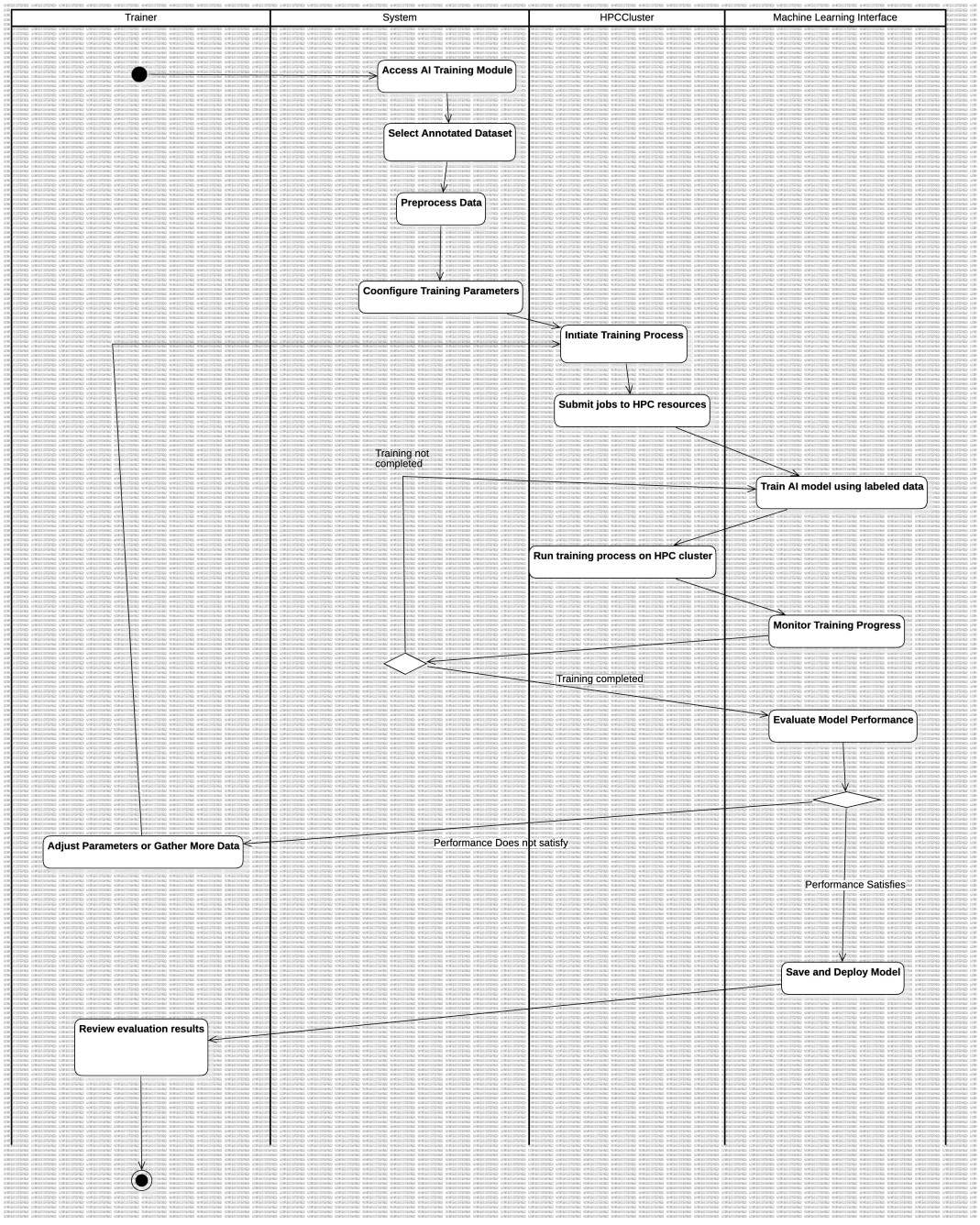

Figure 3.4: Activity Diagram for Train AI Model for Species Identification

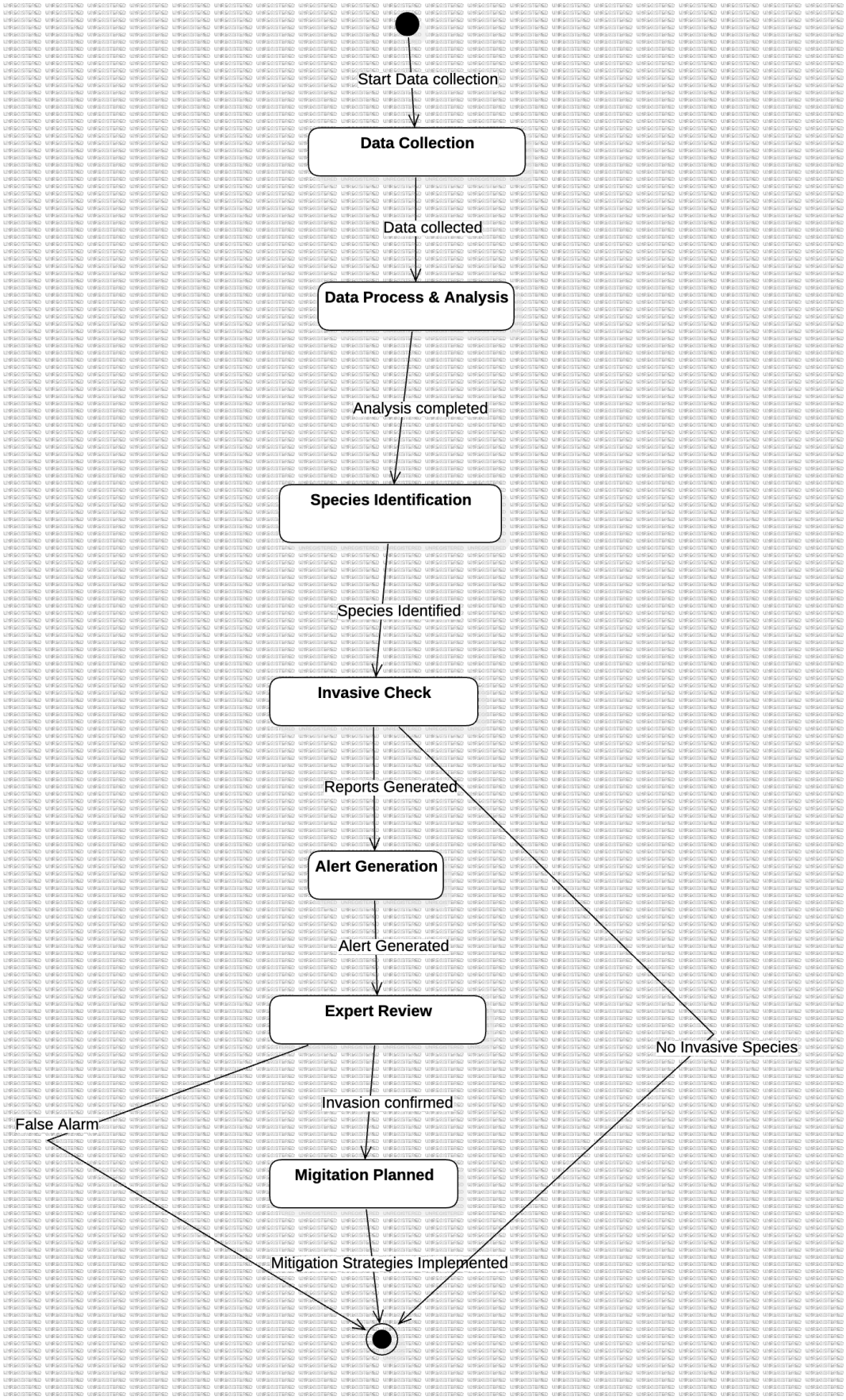

Figure 3.5: State Diagram for Generate Reports on Species Trends

3.3 Logical Database Requirements

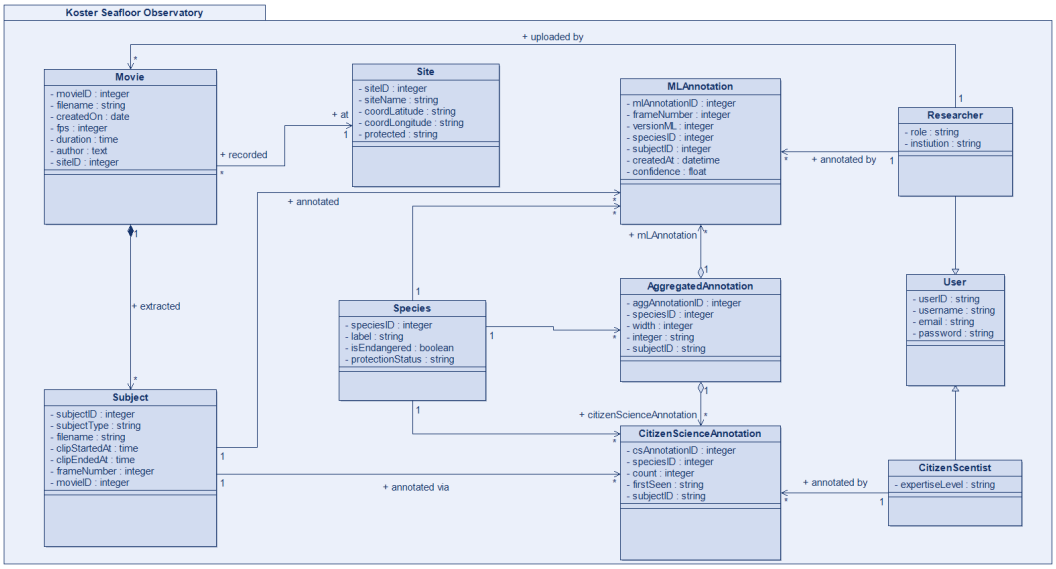

Figure 3.6: Class Diagram For Logical Database Representation

3.4 System Quality Attributes

The Koster Seafloor Observatory (KSO) software system has been designed with a strong emphasis on quality attributes that directly affect both user satisfaction and system robustness. The following quality attributes are prioritized in order of importance:

- **Usability:** The system shall provide an intuitive and accessible interface for its varied user groups—citizen scientists, researchers, data scientists, and project coordinators. The system should aim for ease of use with effective and efficient task completion. Specific usability requirements include:
  - Users should be able to complete common tasks (e.g., annotating video footage or retrieving analysis reports) with minimal error. The average time to perform an annotation task should not exceed 60 seconds for a new user, improving as they gain experience.
  - Response time for user actions (e.g., annotation submission or data retrieval) must be under 2 seconds on standard hardware for at least 95 percent of transactions.
  - User satisfaction should meet a target of at least 85 percent on usability and ease-of-use indices based on periodic surveys.

  - The interface must conform to accessibility guidelines (e.g., WCAG 2.1 AA), ensuring usability for people with disabilities.

- **Performance:** Given the large volume of high-resolution underwater footage and intensive processing involved, performance is a critical quality attribute. Specific performance requirements include:

  - 95% of interactive transactions (e.g., video upload acknowledgment, annotation submission, and query responses) should complete within 1 second under normal workload conditions.
  - The system must support up to 100 simultaneous active users without noticeable performance degradation, with scalability provisions to handle peak periods.
  - The data management component must handle high-resolution video data (from 720p to 4K), with file sizes up to 5 GB, supporting up to 100 GB/hour throughput during peak operations.
  - Dynamic tasks such as model inference on stored video segments should complete within an average processing time of 2–3 seconds per image.

- **Dependability:** Dependability properties, including reliability, availability, and security, are essential to ensure system robustness. Specific dependability requirements include:

  - **Reliability:** The system must maintain at least 99.9% uptime over a 30-day period. Critical component failures should trigger self-healing mechanisms (e.g., automated failover and recovery processes), with service restoration within 5 minutes.
  - **Availability:** Redundancy measures, checkpointing, and disaster recovery procedures should ensure the system is always available. The system must resume operations within 2 minutes following unplanned interruptions or planned maintenance.
  - **Security:** Sensitive data, such as location data and user-generated annotations, must be protected from unauthorized access. Encryption (TLS 1.3 for communications), robust authentication mechanisms, and detailed logging practices should ensure privacy and data integrity. The system must undergo periodic vulnerability assessments.

- **Maintainability:** The system's maintainability is ensured through modularity, clear coding practices, and ease of updates. Specific requirements include:

  - The system architecture should be modular, utilizing containerization (e.g., Docker Compose) and a microservices approach to minimize the impact of updates on overall functionality.

  - At least 95% of the code should be well-documented and adhere to established coding standards to support debugging, maintenance, and future enhancements.
  - Changes should be achievable with a mean time to repair (MTTR) of under 4 working hours per incident.
  - The system must be platform-agnostic, with no more than 10% of the codebase being platform-dependent, facilitating deployment across various environments (e.g., local, cloud, or HPC).

These attributes ensure that the KSO system is user-friendly, high-performing, dependable, and maintainable, providing an optimal experience for all users while also being scalable and secure for the project's long-term success.

3.5 Design Constraints

The design of the Koster Seafloor Observatory (KSO) software system is subject to a number of external constraints imposed by standards, regulatory requirements, and project-specific limitations. These constraints influence architectural decisions, technology selections, and overall system functionality. The key design constraints include:

- **Standards Compliance:**
  - The system shall conform to relevant international standards such as IEEE 29148-2018 for requirements documentation and ISO/IEC quality and security standards.
  - The design must adhere to best practices defined in industry standards for software design and data handling.
- **Regulatory Requirements:**
  - The KSO software must comply with applicable local, regional, and international regulations, including data privacy laws (e.g., GDPR) and research ethics guidelines.
  - Sensitive information (e.g., geolocation data, user annotations) must be handled in accordance with these regulatory frameworks.
- **Security and Data Privacy:**
  - Robust security measures must be implemented to safeguard data in transit and at rest, following industry-standard cryptographic protocols (e.g., TLS 1.3).

  - The system design shall include secure logging and auditing features to ensure data integrity and traceability.

- **Technology Limitations:**

  - The system must integrate with existing hardware (e.g., underwater camera systems) and software components (e.g., cloud storage, HPC clusters) that have their own technical limitations.
  - The design must account for the processing and storage requirements of high-resolution underwater video data.

- **Project Constraints:**

  - Budget, time, and resource limitations may restrict the scope and feature set available in the initial system release.
  - The design must emphasize scalability, modularity, and maintainability to facilitate future enhancements without extensive rework.

These design constraints serve as guiding principles in the architectural and technical decision-making process, ensuring that the KSO system is both compliant with external requirements and optimized for the project's operational context.

3.6 Supporting Information

This section provides additional details that enhance the understanding of the software system beyond its core functional requirements. This information includes sample data formats, cost considerations, and results from user surveys that validate the system's effectiveness. Additionally, it outlines relevant background details, contextual information, and the specific challenges the software aims to address. Lastly, it specifies any special packaging requirements, ensuring proper deployment and adherence to security and data-sharing standards. These elements collectively support the clarity, usability, and implementation of the system.

The following supporting information is provided to enhance the understanding of the system:

- **Sample Input/Output Formats, Cost Analysis, and User Surveys**

  - The system processes underwater video footage obtained from remotely operated vehicles (ROVs) and autonomous underwater vehicles (AUVs). Sample input formats include high-definition MP4 and MOV video files.

  - The processed output consists of annotated video clips, species classification data, and machine learning predictions stored in SQLite databases following Darwin Core (DwC) standards.
  - Cost analysis includes the infrastructure costs for data storage (long-term and short-term servers), high-performance computing (HPC) resources, and citizen science platform maintenance.
  - A user survey was conducted to evaluate the accuracy of citizen scientists in species classification. Aggregated results indicate an 80

- **Background and Supporting Information**
  - The software is an open-source, modular system for analyzing subsea movie data. It integrates three primary modules: data management, citizen science classification, and machine learning-based species identification.
  - The system is hosted on the Zooniverse citizen science platform, where volunteers annotate footage to train machine learning models.
  - High-performance computing resources facilitate training and deploying deep learning models, allowing real-time classification of marine species.

- **Description of the Problem to be Solved**
  - Marine ecological research requires large-scale analysis of underwater imagery to monitor biodiversity and habitat changes.
  - Manual annotation of subsea movies is time-consuming and requires expert knowledge, limiting scalability.
  - The software provides an automated and scalable approach by combining citizen science and machine learning to extract ecological insights from video data.

- **Special Packaging Instructions**
  - The software is developed using open-source technologies and is available via GitHub repositories for data processing and machine learning.
  - The machine learning model is deployed as a FastAPI-based web service, providing a user-friendly interface for researchers.
  - The SQLite database and API ensure compatibility with existing biodiversity data portals.
  - Security measures include access control for sensitive datasets and compliance with international biodiversity data-sharing standards.

4. Suggestions to Improve The Existing System

4.1 System Perspective

The KSO (Knowledge Sharing Organization) system facilitates knowledge exchange between researchers, citizen scientists, and external databases while integrating machine learning capabilities. The following context diagram provides an overview of the external entities interacting with KSO.

Explanation of the Context Diagram: The diagram illustrates the interactions between the KSO system and external entities, highlighting data flow and system boundaries. The more detailed explanations for the existing entities already provided in the first section. However, The updated KSO context diagram introduces **Organizations** as a key external entity, enabling organizations to work closely with KSO system in order to share critical data.

Entities and Interactions

- **KSO (Central System)**: Manages data collection, processing, and distribution.
- **Researchers**: Provide research data and receive processed insights.
- **Citizen Scientists**: Contribute field observations and receive analyzed data.
- **Machine Learning System**: Enhances data processing through pattern recognition and AI-driven insights.
- **External Databases**: Provide structured and unstructured datasets to KSO.
- **Organizations**: Accesses the critical data provided by KSO. By checking reports and alerts, they intend to take action.

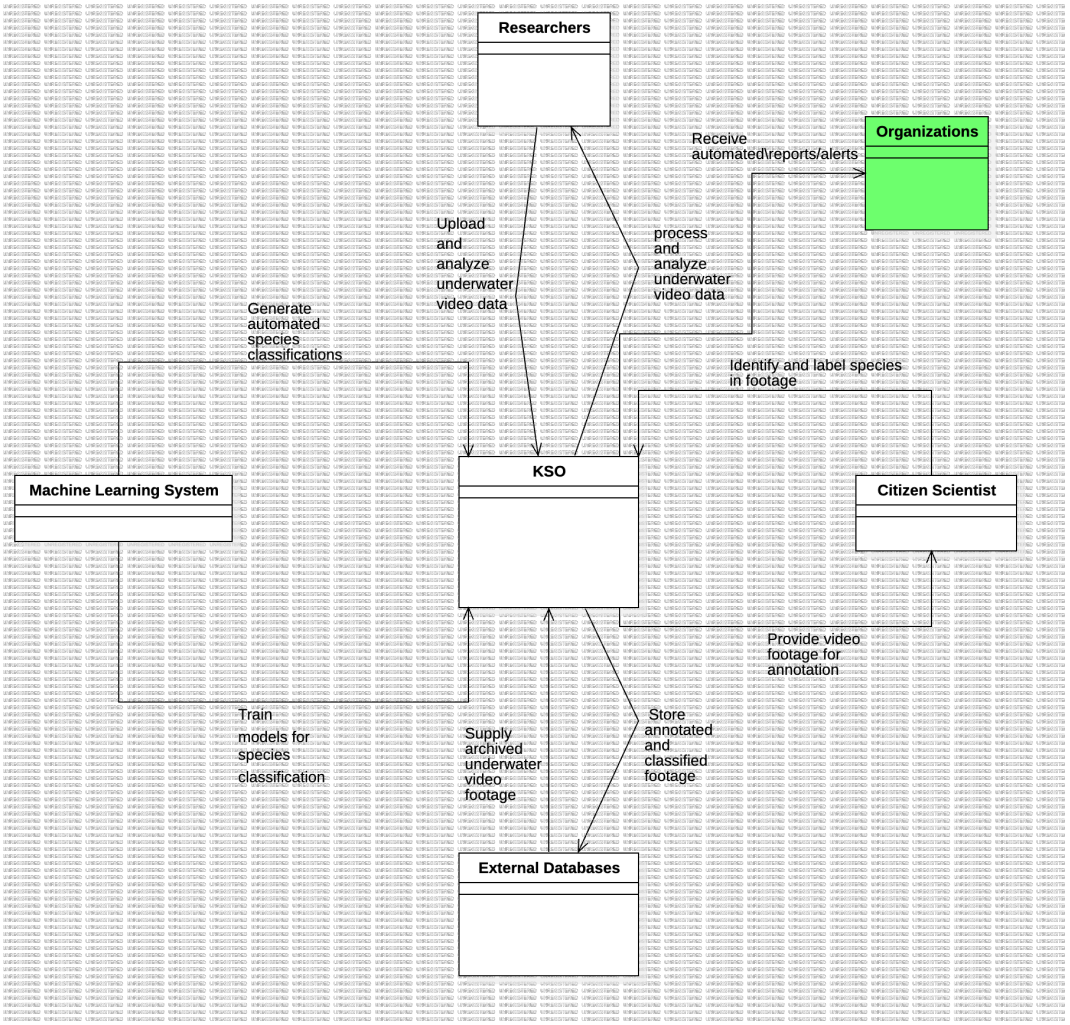

Figure 4.1: Suggested Context Diagram

Data Flow

- Researchers submit research data to KSO and retrieve relevant processed insights.
- Citizen Scientists contribute observational data, which KSO processes and returns as structured information.
- The Machine Learning System improves data quality and generates predictive analytics.
- KSO communicates with External Databases to fetch and store relevant datasets.

Improvements to the Existing System

The updated KSO context diagram introduces **Organizations** as a key external entity, improving system efficiency through real-time data sharing, automation, and security enhancements. The key improvements include:

- **Automated Reports and Alerts:** Organizations now receive real-time automated alerts based on analyzed underwater footage.
- **Real-Time Data Processing:** The system enhances streaming capabilities, ensuring organizations can act on ecological changes immediately.
- **AI-Driven Insights:** Machine learning models provide predictive analytics and automated risk assessments, improving decision-making.
- **Improved Security and Access Control**: Use Role-Based Access Control (RBAC) for better data governance.
- **Better Integration with External Databases:** Organizations can access classified and annotated data, enhancing collaboration.
- **Improved Security and Access Control:** Role-Based Access Control (RBAC) ensures that organizations receive only relevant reports and insights.
- **User-Friendly Interfaces**: Develop an improved UI/UX for easier interaction with the system.

These enhancements strengthen the KSO system by making it more responsive, intelligent, and accessible to key stakeholders.

4.2 External Interfaces

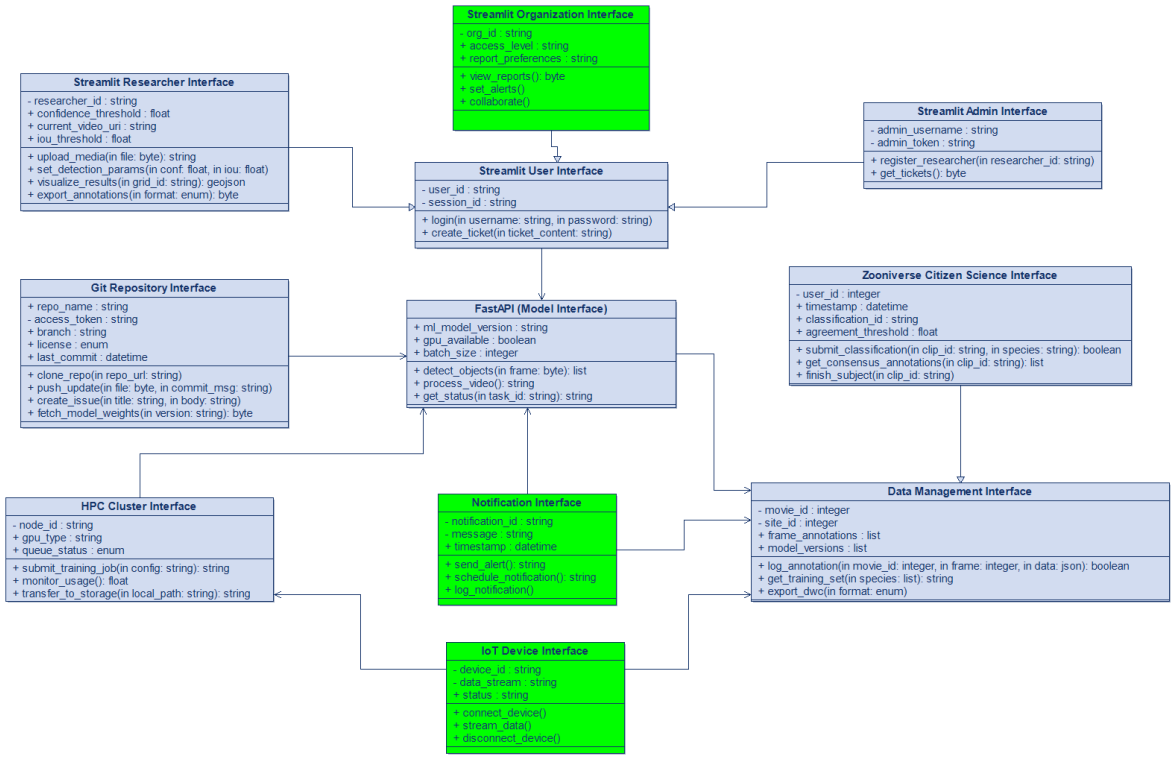

Figure 4.2: Suggested External Interfaces Diagram

Explanations for newly added interfaces:

IoT Interface Purpose: Facilitates the integration of real-time data from IoT devices into the system for continuous monitoring and data collection.

Attributes:

- `device_id`: Unique identifier for each IoT device.
- `data_stream`: Continuous data input from sensors.
- `status`: Current operational status of the device.

Operations:

- `connect_device()`: Establish a connection with the IoT device.
- `stream_data()`: Continuously send data to the system.
- `disconnect_device()`: Safely terminate the connection.

Notification Interface Purpose: Keeps users informed about important events and updates within the system through timely alerts and notifications.

Attributes:

- `notification_id`: Unique identifier for each notification.
- `message`: Content of the notification.
- `timestamp`: Time when the notification is sent.

Operations:

- `send_alert()`: Dispatch a notification to users.
- `schedule_notification()`: Set up future notifications based on events.
- `log_notification()`: Record notification details for auditing.

Streamlit Organizations Interface Purpose: Provides conservation organizations with tailored access to the system, enabling them to view reports, set alerts, and collaborate.

Attributes:

- `org_id`: Identifier for the organization.
- `access_level`: Permissions granted to the organization.
- `report_preferences`: Preferences for receiving reports and notifications.

Operations:

- `view_reports()`: Access detailed ecological reports.
- `set_alerts()`: Configure alerts for specific events or data changes.
- `collaborate()`: Engage with other organizations and researchers.

4.3 Functions

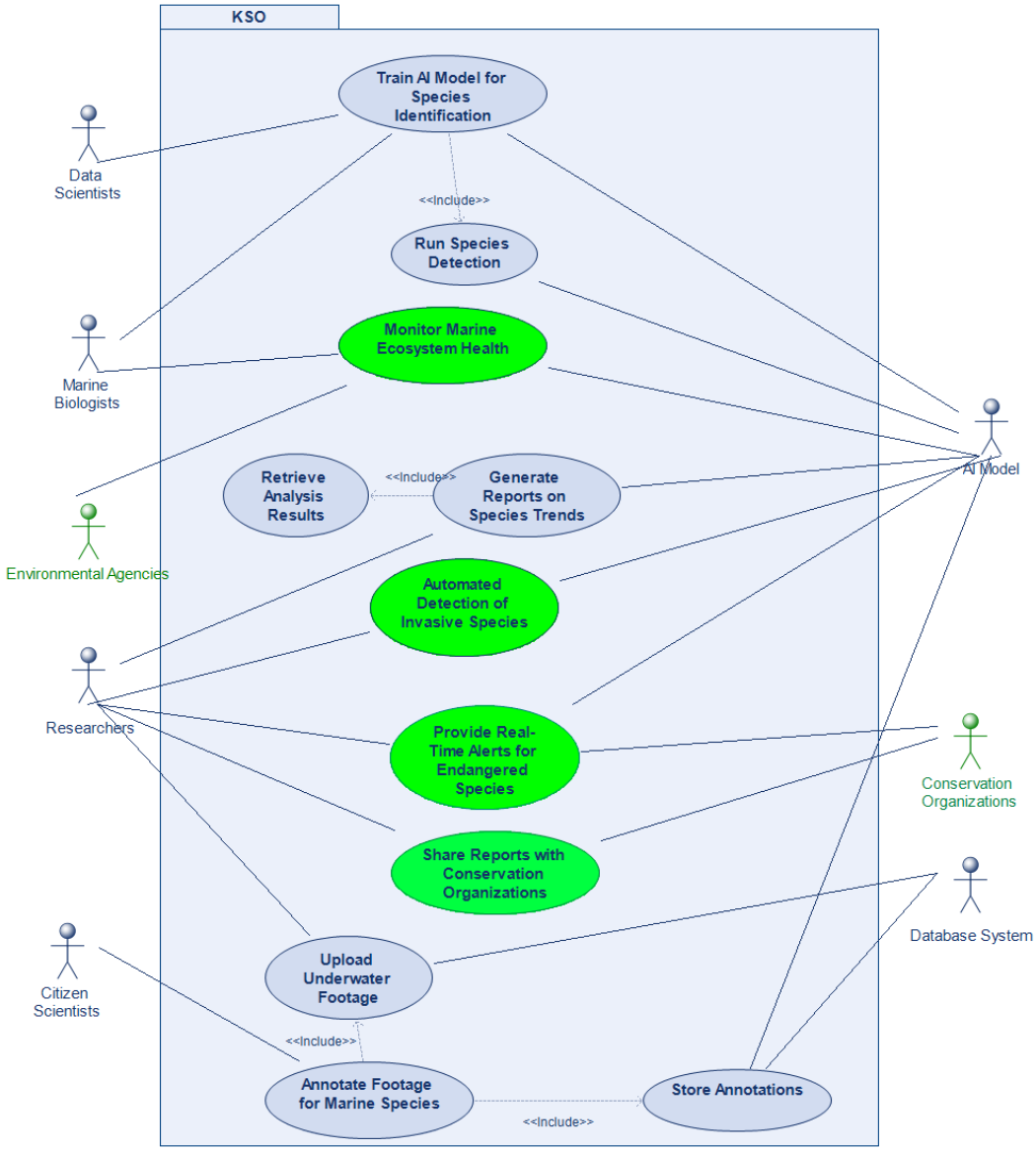

Figure 4.3: Suggested Use Case Diagram

Table 4.1: Use Case: Automated Detection of Invasive Species

Use Case Name	**Automated Detection of Invasive Species**
Actors	AI Model, Researchers
Description	The system processes collected marine footage and sensor data to detect invasive species. If an invasive species is identified, alerts are generated for researchers and biologists for further investigation.
Pre-Conditions	- Footage and sensor data must be available. - AI model must be trained on existing species data. - Researchers must have access to review detected species.
Data	- Collected marine footage and sensor data - AI model detection logs - Species database (native and invasive) - Researcher feedback logs
Response	- System processes new footage and sensor data. - System identifies potential invasive species. - System notifies researchers if invasive species are detected. - Researchers review the detected species and validate the finding. - System updates detection models based on researcher input.
Stimulus	- New footage and sensor data are uploaded. - AI model processes new data for detection. - Researchers receive an alert for detected invasive species.
Normal Flow	1. System collects and processes marine footage. 2. AI model analyzes data and identifies species. 3. System cross-checks identified species with the invasive species database. 4. If an invasive species is detected, the system generates an alert. 5. Researchers review the detected species and validate the finding. 6. System updates detection accuracy based on researcher feedback. 7. If validated, the system stores data for future monitoring.

Continued on next page

Table 4.1 – continued from previous page

Use Case Name	**Automated Detection of Invasive Species**
Alternative Flow	- If the AI model is uncertain, it flags species for manual review by experts. - If an invasive species is detected but not in the database, researchers can manually classify and update records.
Exception Flow	- If footage upload fails, system prompts retry. - If AI model confidence is low, species are flagged for manual review. - If false positives are detected, the model is retrained to improve accuracy.
Post-Conditions	- Invasive species detections are recorded and stored. - Researchers validate and refine AI model results. - The system updates its species recognition model based on expert input.
Comments	- System periodically updates the invasive species list based on new findings. - Researchers provide feedback to refine detection accuracy. - Alerts are prioritized based on potential ecological impact.

Table 4.2: Use Case: Monitor Marine Ecosystem Health

Use Case Name	**Monitor Marine Ecosystem Health**
Actors	Marine Biologists, Conservation Organizations, AI Model
Description	AI analyzes footage and sensor data to assess ecosystem health, detecting patterns in species behavior, population density, and habitat conditions.
Pre-Conditions	- Footage and sensor data must be collected. - AI model must be trained on ecosystem health indicators. - Environmental agencies require real-time access to insights.
Data	- Marine footage and sensor data - Species population trends - Water quality and temperature data - AI model analysis results

Continued on next page

Table 4.2 – continued from previous page

Use Case Name	**Monitor Marine Ecosystem Health**
Response	- System analyzes data and generates ecosystem health metrics. - System provides visualization tools for trends. - Alerts issued if significant changes are detected.
Stimulus	- New data is uploaded or streamed from sensors. - AI model detects significant deviations. - Researchers request health assessments.
Normal Flow	1. Sensors and cameras capture ecosystem data. 2. System processes data and extracts relevant features. 3. AI model analyzes species distribution, water quality, and environmental changes. 4. System generates reports and alerts. 5. Researchers and agencies review data.
Alternative Flow	- If AI model confidence is low, expert review is triggered. - Manual data annotation is conducted for verification.
Exception Flow	- If sensor data is missing, system alerts users. - If AI misclassifies data, retraining is required.
Post-Conditions	- Ecosystem health reports generated. - Anomalies flagged for further investigation. - Data stored for long-term trend analysis.
Comments	- Researchers can compare data across different regions and time periods. - AI continuously improves its predictions with new data.

Table 4.3: Use Case: Share Reports with Conservation Organizations

Use Case Name	**Share Reports with Conservation Organizations**
Actors	Researchers, Conservation Organizations
Description	Researchers generate and share ecological reports with conservation organizations to support biodiversity monitoring and policy-making.

Continued on next page

Table 4.3 – continued from previous page

Use Case Name	**Share Reports with Conservation Organizations**
Pre-Conditions	- Reports are generated and validated. - Conservation organizations have access permissions.
Data	- Species trend reports - Ecological impact assessments - Conservation recommendations
Response	- Reports are shared via secure channels. - Confirmation of receipt is logged.
Stimulus	Request from conservation organizations or scheduled sharing interval.
Normal Flow	1. Researchers compile and validate reports. 2. System checks access permissions for organizations. 3. Reports are securely transmitted to organizations. 4. Receipt confirmation is logged. 5. Organizations review and utilize reports for conservation efforts.
Alternative Flow	- If access permissions are not valid, system requests authorization.
Exception Flow	- If transmission fails, system retries and notifies researchers.
Post-Conditions	- Reports are successfully shared and logged. - Feedback from organizations is recorded for future improvements.
Comments	- Reports can be shared in multiple formats (e.g., PDF, CSV). - Regular feedback loops with organizations enhance report quality.

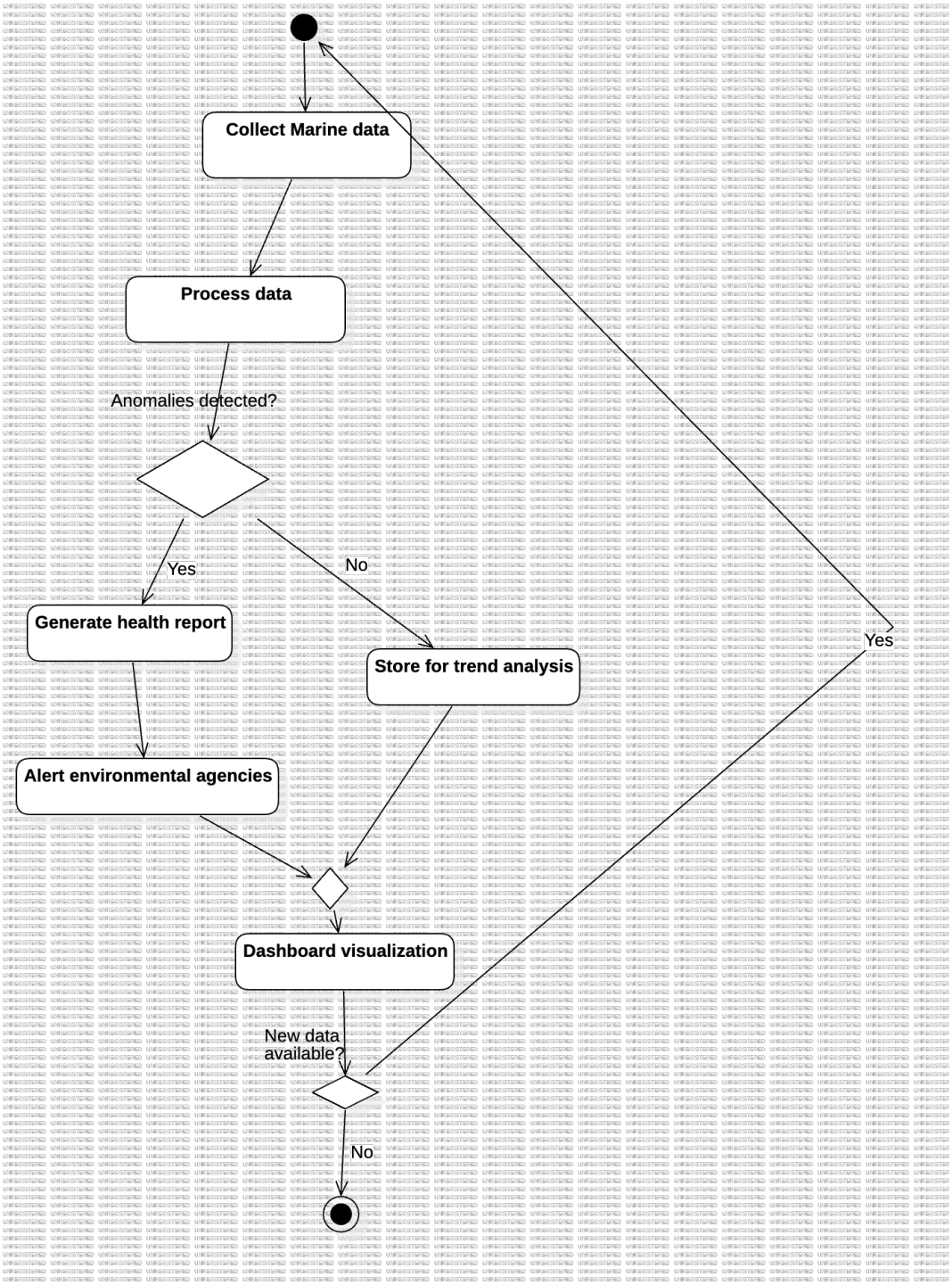

Figure 4.4: Activity Diagram of Monitoring Marine Ecosystem Health

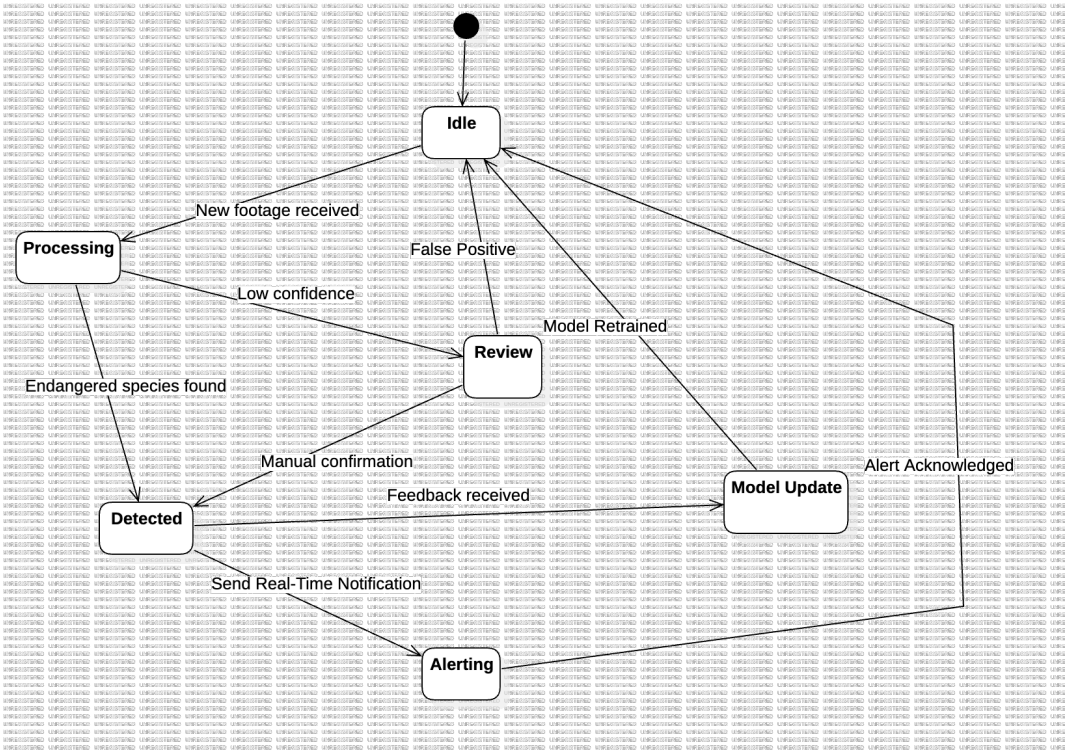

Figure 4.5: State Diagram of Providing Real-Time Alerts for Endangered Species

Table 4.4: Use Case: Provide Real-Time Alerts for Endangered Species

Use Case Name	**Provide Real-Time Alerts for Endangered Species**
Actors	Conservationists, Researchers, AI Model
Description	AI model detects and alerts conservationists about sightings of endangered species in real-time to enable prompt protective actions.
Pre-Conditions	- System must have access to real-time footage and sensor data. - AI model must be trained to recognize endangered species. - Conservationists need access to alert notifications.
Data	- Live camera feeds - Detected endangered species data - Location and timestamp of sightings - Historical presence records
	Continued on next page

Table 4.4 – continued from previous page

Use Case Name	**Provide Real-Time Alerts for Endangered Species**
Response	- System sends alerts when endangered species are detected. - Conservationists receive species location and timestamp. - System logs data for future reference.
Stimulus	- AI model identifies an endangered species. - Conservationists receive real-time notifications. - Authorities may take protective measures.
Normal Flow	1. Live feeds and sensors continuously monitor habitats. 2. AI model processes video and identifies species. 3. If an endangered species is detected, an alert is triggered. 4. System records details and sends notifications. 5. Conservationists act on alerts and log findings.
Alternative Flow	- If AI confidence is low, human verification is required. - Users can submit manual confirmations or corrections.
Exception Flow	- If real-time data is unavailable, system notifies users of gaps. - False positives are flagged for model retraining.
Post-Conditions	- Sightings of endangered species are recorded. - Conservationists receive timely notifications. - AI model improves through continued learning.
Comments	- Alerts can be customized by species or region. - AI continuously adapts to improve detection accuracy. - Conservation efforts can be coordinated more effectively.

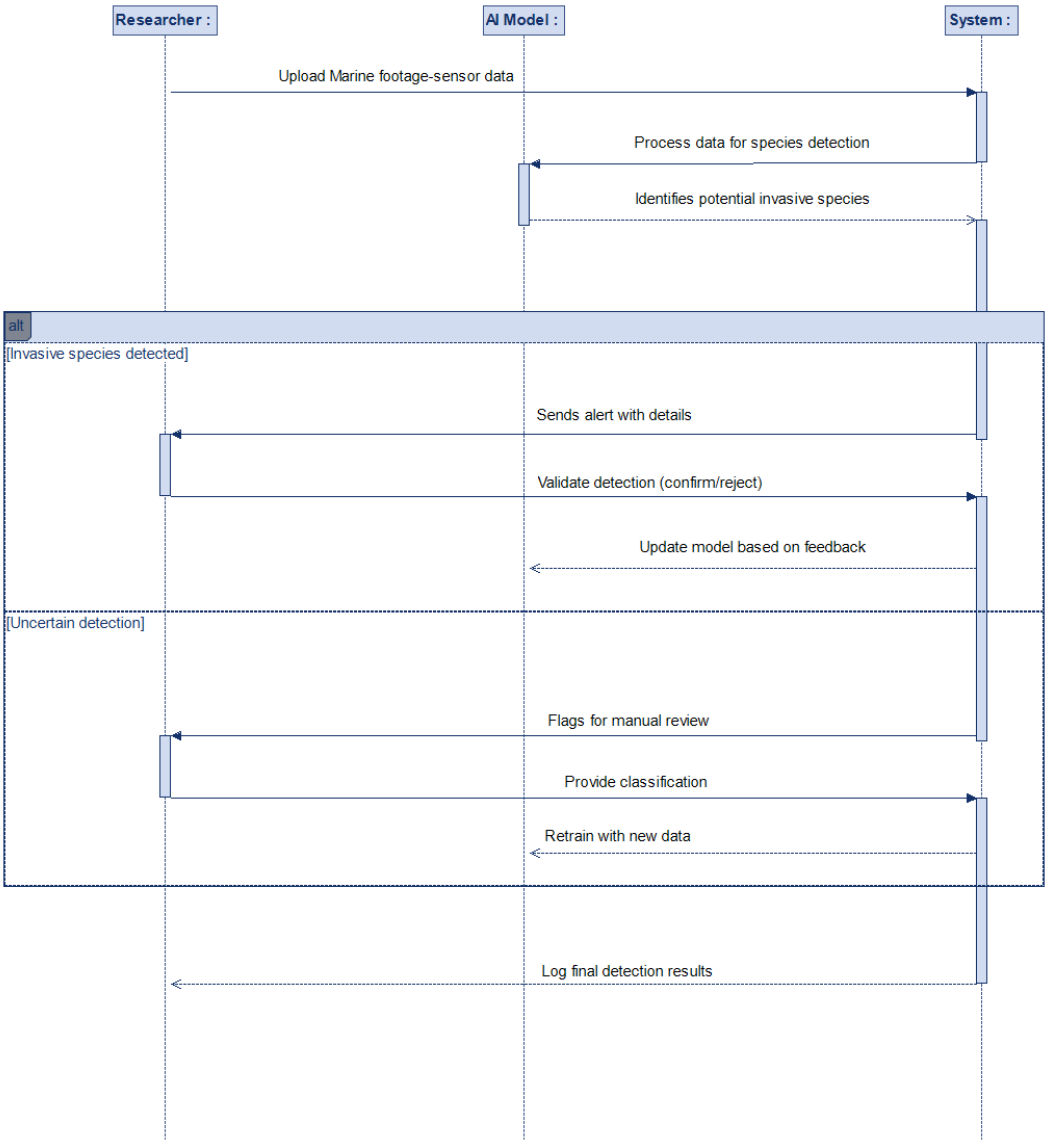

Figure 4.6: Sequence Diagram of Automated Detection of Invasive Species

4.4 Logical Database Requirements

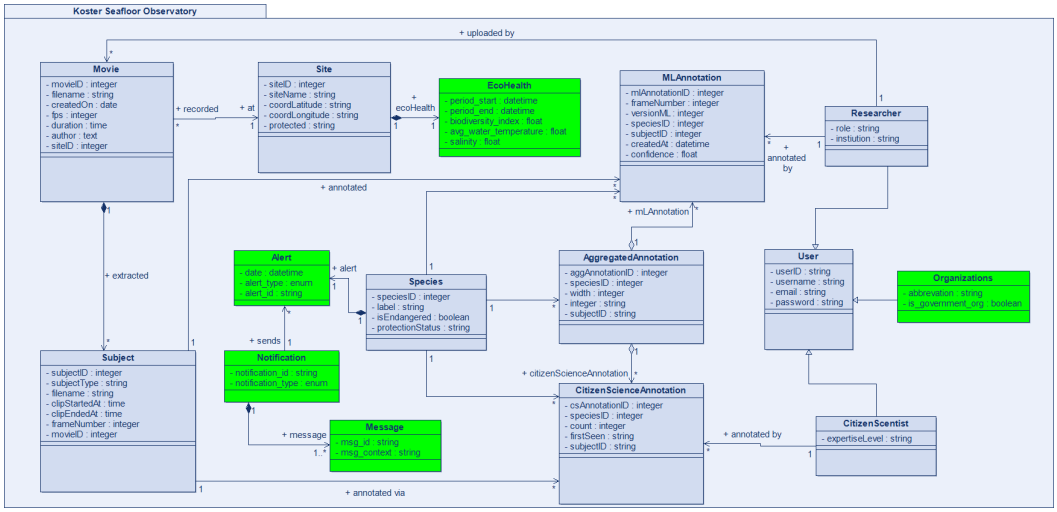

Figure 4.7: Suggested Logical Database Requirements

4.5 System Quality Attributes

The enhanced Koster Seafloor Observatory (KSO) system prioritizes the following quality attributes to ensure robustness, scalability, and user satisfaction. These are ranked by importance with measurable requirements:

4.5.1 Usability

Goal: Ensure intuitive interaction for diverse users (citizen scientists, researchers, developers).

- **Learnability:**
  - New users (citizen scientists) should complete annotation tasks within **45 seconds** (improved from 60s) after minimal training.
  - Researchers should generate custom reports in $\leq$**3 clicks** via the Streamlit interface.
- **Efficiency:**
  - 95% of user interactions (e.g., uploading footage, submitting annotations) must respond in $<$**1 second**.

  - AI-assisted annotation suggestions should reduce manual labeling time by **40%**.

- **Accessibility:**
  - Conform to **WCAG 2.1 AA** standards (e.g., screen-reader compatibility, high-contrast UI).
  - Provide multilingual support (English, Swedish) for citizen scientists.

- **User Satisfaction:**
  - Achieve ≥90% satisfaction in post-use surveys (up from 85%).

4.5.2 Performance

Goal: Handle large-scale data processing with minimal latency.

- **Throughput:**
  - Support **200 simultaneous users** (up from 100) during peak loads (e.g., citizen science campaigns).
  - Process **4K video footage** at **5 GB/min** (compressed) with parallel HPC/cloud processing.

- **Latency:**
  - Real-time alerts for endangered species must trigger within **500 ms** of detection.
  - Batch processing (e.g., model training) should complete **50% faster** via optimized GPU utilization.

- **Scalability:**
  - Dynamically scale cloud resources (AWS/GCP) to handle **10x** data volume spikes.

4.5.3 Dependability

Goal: Ensure fault tolerance and data integrity.

- **Reliability:**
  - Maintain **99.99% uptime** (improved from 99.9%) with automated failover for critical services (e.g., FastAPI inference servers).

  - Self-healing mechanisms (e.g., container restart) should recover from failures in ≤**2 minutes**.

- **Security:**
  - Encrypt all sensitive data (GPS coordinates, user info) with **AES-256** and enforce **TLS 1.3** for APIs.
  - Implement **RBAC** (Role-Based Access Control) with biometric authentication for admin roles.

- **Data Integrity:**
  - Use **blockchain-based hashing** for annotation logs to prevent tampering.

4.5.4 Maintainability

Goal: Simplify updates and debugging.

- **Modularity:**
  - Containerize 100% of components (Docker) with microservices architecture.
  - Limit platform-dependent code to ≤**5%** (down from 10%).

- **Documentation:**
  - Provide auto-generated Swagger docs for APIs and Jupyter Notebook tutorials for ML workflows.

- **MTTR (Mean Time to Repair):**
  - Resolve critical bugs in ≤**2 hours** (improved from 4 hours) via CI/CD pipelines.

4.6 Design Constraints

4.6.1 Regulatory Compliance

- **GDPR/CCPA:** Anonymize citizen scientist data and obtain explicit consent for public datasets.
- **Marine Research Ethics:** Adhere to **Nagoya Protocol** for biodiversity data sharing.

4.6.2 Hardware/Software Limitations

- **Camera Systems:** Support legacy ROV/AUV formats (e.g., **DeepSea Power & Light** cameras) with **RTSP/ONVIF** protocols.
- **ML Frameworks:** Limit dependencies to **PyTorch 2.0+** and **TensorFlow Lite** for edge deployment.

4.6.3 Interoperability

- **APIs:**
  - RESTful APIs must follow **OpenAPI 3.0** standards for integration with Zooniverse and GBIF.
  - Real-time data streaming via **MQTT** for sensor networks.
- **Data Formats:**
  - Videos: **MP4 (H.265)** or **MOV (ProRes)**.
  - Annotations: **Darwin Core (DwC)** for biodiversity databases.

4.6.4 Project Constraints

- **Budget:** Optimize cloud costs using **spot instances** for non-critical ML training.
- **Time:** Phase deployments (e.g., real-time alerts first, then predictive analytics).

4.6.5 Ethical & Environmental

- **Carbon Footprint:** Prioritize **Green AI** practices (e.g., quantized models for low-energy inference).
- **Bias Mitigation:** Audit ML models for **fairness** (e.g., equal detection accuracy across species).

4.7 Supporting Information

The proposed improvements to the Koster Seafloor Observatory (KSO) system are designed to significantly enhance its functionality and usability, thereby increasing its impact on marine biodiversity

research. By integrating advanced machine learning capabilities, the system will offer AI-driven insights that predict trends and automate reporting, providing researchers with powerful analytical tools. Real-time data processing and alerts will enable immediate insights and actions, crucial for monitoring endangered species and responding promptly to ecological changes. Enhanced data validation and security measures, including role-based access control, will ensure data integrity and secure access, fostering trust among users. Additionally, the development of user-friendly interfaces and sharing data/reports will increase accessibility and engagement for a diverse range of users, including researchers, citizen scientists, and conservation organizations. These enhancements will facilitate more efficient data collection, processing, and sharing, ultimately contributing to more effective marine conservation efforts. By fostering collaboration and leveraging cutting-edge technology, the KSO system will continue to be a valuable tool in understanding and preserving marine ecosystems.